M'Hamed Bougara University of Boumerdes **2026-2027**

Faculty of Sciences – Computer Science Department

Promotion : M1 – LMD. **Module : Apprentissage automatique**

Pr. AR. Maouche

Lab 2: Data cleaning, normalization and encoding

1- Objectives of the lab

1- Dataset cleaning principle operations : how to drop unrequired rows and variables, how to drop duplicates and handle missing data.
2- Introduction to data normalization, objectives and some techniques
3- Introduction to data encoding, objectives and some techniques

The dataset

For this lab, we will use (students.csv), a simple dataset containing 40 rows about students, which includes duplicates, missing values, irrelevant data (like negative ages), and fields requiring normalization and encoding. The dataset is in CSV format and contains the following information : Name,Age,Gender,Score,Grade,City.

Data cleaning Execute the following :

1- Import the basic libraries ;
2- Load the dataset and show the first rows (head) ;
3- **Check for duplicates and remove them** using the following pandas functions :

```
dflab2.duplicated()

duplicates = dflab2.duplicated()

print(dflab2[duplicates])

print(f"Number of duplicate rows: {duplicates.sum()}")

dflab2_cleaned = dflab2.drop_duplicates()

dflab2.drop_duplicates(inplace=True)
```

- **duplicated()** returns a Boolean series, where True indicates a duplicate row.
- **dflab[duplicates]** will display all the rows that are duplicates.
- **duplicates.sum()** gives the total number of duplicate rows.
- **drop_duplicates()** removes the duplicaqted data, optionnally: create a new dataframe or use the same.

4- Check for **missing data** and process them.

```
missing_values = dflab2.isnull().sum()

print(missing_values)
```

Ways to handle missing data:

Remove missing data: Remove the rows with missing data (if only a small portion of the data is missing) (**dropna**() function)

Imputation: Replace missing values with statistical measures like the mean (**fillna function**)

- Try the following options and check for the resulting dataset :

```
dflab2['Score'] = dflab2['Score'].fillna(dflab2['Score'].mean()))

print(dflab2.isnull().sum())

dflab2= dflab2.dropna()

print(dflab2.isnull().sum())
```

Data normalization

The objective of data **normalization** is to scale the data into a specific range, often between 0 and 1. This is especially important when the dataset contains features that have different ranges for example age (ranging from 0 to 100) and income (ranging from 10,000 to 100,000) to avoid the dominance of features woth larger values.

Min-Max Scaling: This scales the data into a fixed range, usually between 0 and 1 according to the formula :

$$X_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Example :

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

dflab2['Age'] = scaler.fit_transform(dflab2[['Age']])

print(dflab2.head())

dflab2['Age'] = scaler.fit_transform(dflab2[['Age']])

print(dflab2.head())
```

Scikit-learn (sklearn) is an open-source Python library designed for machine learning and data mining. It provides a wide range of machine learning models, including those for regression, classification, and clustering, as well as tools for model selection and preprocessing.

Data encoding

Encoding is process of converting categorical values (like strings or gender) into numerical values in a dataset . This can be done through techniques like **label encoding** or **one-hot encoding**.

- **Label Encoding** assigns a numerical label to each category (useful for ordinal data). For example, giving the values Low, Medium, High, assign to them the following label : 0, 1 ,2.

- **One-Hot Encoding** creates new binary columns for each category (useful for non-ordinal data). For example, if you have a column with values "Red", "Blue", and "Green", you will get three new columns: "Red" => [1, 0, 0], "Blue" => [0, 1, 0], "Green" => [0, 0, 1].

Execute the following example

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

dflab2['Gender'] = encoder.fit_transform(dflab2['Gender'])

print(dflab2.head())

df_one_hot = pd.get_dummies(dflab2, columns=['City'], drop_first=True)

print(df_one_hot)
```